MPOO 2027-1 · GRUPO 6 · PRÁCTICA 7 · MIÉRCOLES 14 DE OCTUBRE

Herencia, primera parte

Dos jerarquías de tres niveles, cada clase en su archivo. Primero `Animal`, junto con el profesor; después `Personaje`, por tu cuenta. En las dos usas lo del lunes: `extends`, `super(...)` y `@Override`.

Descargar los archivos (.zip)

En una línea

Escribes ocho clases. Terminas cuando `MainAnimales` y `MainPersonajes` imprimen **exactamente** lo que dice esta guía.

ARCHIVO	QUÉ ES	QUÉ HACES
`MainAnimales.java`	El `main` de la parte 1	Correrlo. No se toca
`Animal`, `Perro`, `PerroGuia`, `Gato`	Parte 1, con el profesor	Escribirlas
`MainPersonajes.java`	El `main` de la parte 2	Agregarlo **cuando empieces la parte 2**. No se toca
`Personaje`, `Caballero`, `Paladin`, `Mago`	Parte 2, por tu cuenta	Escribirlas

Todo va **junto en la misma carpeta** (en IntelliJ, en `src`). `MainPersonajes.java` se agrega hasta la parte 2 porque IntelliJ compila todo lo que hay en `src`: si lo pones antes, la parte 1 no corre, porque `Personaje` todavía no existe.

Parte 0 · Qué se entrega y cómo se califica

QUÉ	DETALLE
Cuándo	**Viernes 16 de octubre, 23:59.** Se empieza en el laboratorio y ahí mismo se resuelven las dudas. Sin IA, como siempre.
Con quién	En parejas, o de tres si alguien se queda sin pareja. Se reparten el trabajo como quieran, pero **cada integrante sube la entrega completa en su propia carpeta**.
Dónde	Tu rama `entregas_apellido_nombre`, carpeta `entregas/apellido_nombre/p07/`. Solo push, sin pull request.
Qué archivos	Los **diez** `.java` (ocho clases y dos `main`), **todos** los `.class` y una **bitácora en PDF** con: los integrantes (nombre y número de cuenta), la **captura de las dos salidas**, la **parte 3** resuelta y qué se les complicó y cómo lo resolvieron.

Se califica leyendo tu código, sobre 10:

QUÉ REVISO	PUNTOS
Corre y la salida es la de la guía, en las dos partes	3
Cápsula: atributos `private`, sus getters y las validaciones que pide cada constructor	2
Constructores en cadena: cada hijo llama a `super(...)` en su primera línea y no repite los atributos del padre	2
Sobrescritura: `@Override` en cada método sobrescrito, y `super.toString()` y `super.atacar()` donde se piden	2
Bitácora: las dos capturas y la parte 3 explicada con tus palabras	1

El código tiene que ser suyo. Si algo no les sale, entréguenlo igual y cuenten en la bitácora dónde se atoraron.

Parte 1 · Animales, junto con el profesor

```
Animal            nombre, edad          sonido() da "..."
├── Perro         + raza                sonido() da "Guau"
│   └── PerroGuia + asistido            (sonido lo hereda de Perro)
└── Gato          + interior            sonido() da "Miau"
```

El profesor teclea `Animal` y `Perro` en el proyector y tú las tecleas con él. Así quedan:

```java
public class Animal {

    // private: ni sus hijos los tocan
    private String nombre;
    private int edad;

    public Animal(String nombre, int edad) {
        this.nombre = nombre;
        // la cápsula: el constructor valida
        if (edad < 0) {
            this.edad = 0;
        } else {
            this.edad = edad;
        }
    }

    public String getNombre() {
        return nombre;
    }

    public int getEdad() {
        return edad;
    }

    public String sonido() {
        return "...";                  // un animal a secas
    }

    // sobrescribe el toString de Object
    @Override
    public String toString() {
        return nombre + ", edad " + edad;
    }
}
```

```java
// un Perro ES UN Animal
public class Perro extends Animal {

    // solo lo nuevo: nombre y edad vienen de Animal
    private String raza;

    public Perro(String nombre, int edad, String raza) {
        // SIEMPRE la primera línea: Animal guarda lo suyo
        super(nombre, edad);
        this.raza = raza;               // y luego Perro, lo suyo
    }

    public String getRaza() {
        return raza;
    }

    @Override
    public String sonido() {
        return "Guau";
    }

    @Override
    public String toString() {
        // reutiliza el del padre y le agrega
        return super.toString() + " | " + raza;
    }
}
```

Ahora tú: PerroGuia y Gato

Siguen el mismo patrón. Escríbelas antes de que el profesor las muestre.

CLASE	HEREDA DE	ATRIBUTO NUEVO	CONSTRUCTOR	SOBRESCRIBE

`PerroGuia`	`Perro`	`String asistido`, con su getter	`PerroGuia(String nombre, int edad, String raza, String asistido)`	`toString`: lo del padre más `" | asiste a Marta"`. **`sonido` no**
`Gato`	`Animal`	`boolean interior`, con su getter `isInterior()`	`Gato(String nombre, int edad, boolean interior)`	`sonido`: `"Miau"`. `toString`: lo del padre más `" | de interior"` o `" | de exterior"`

Con las cuatro listas, corre `MainAnimales`:

```
public class MainAnimales {
    public static void main(String[] args) {
        Animal a = new Animal("Lomito", -2);
        Perro p = new Perro("Firulais", 3, "criollo");
        PerroGuia g = new PerroGuia("Max", 5, "labrador", "Marta");
        Gato c = new Gato("Michi", 2, true);

        System.out.println(a + " dice " + a.sonido());
        System.out.println(p + " dice " + p.sonido());
        System.out.println(g + " dice " + g.sonido());
        System.out.println(c + " dice " + c.sonido());
        System.out.println(g.getNombre() + " es " + g.getRaza() + ",
edad " + g.getEdad());
    }
}
```

> **DEBE SALIR**
>
> ```
> Lomito, edad 0 dice ...
> Firulais, edad 3 | criollo dice Guau
> Max, edad 5 | labrador | asiste a Marta dice Guau
> Michi, edad 2 | de interior dice Miau
> Max es labrador, edad 5
> ```

Fíjate en la tercera línea: `PerroGuia` no escribió `sonido()` y aun así dice `Guau`, porque lo hereda de `Perro`, su padre inmediato, no de `Animal`. Y en la última, `g.getNombre()` funciona aunque `PerroGuia` nunca escribió ese método: viene de dos niveles arriba.

Parte 2 · Personajes de videojuego, por tu cuenta

```
Personaje          nombre, vida          atacar() da 10
├── Caballero       + fuerza              atacar() da lo del padre + fuerza
│   └── Paladin     + fe                  atacar() da lo del padre + fe / 2
└── Mago            + mana                atacar() da mana / 2
```

Mismo patrón que los animales, sin código de ejemplo. Todos los atributos son `private` y cada uno tiene su getter (`getNombre()`, `getVida()`, `getFuerza()`, `getFe()`, `getMana()`).

CLASE	CONSTRUCTOR	VALIDA	`ATACAR()`	`TOSTRING()`
`Personaje`	`Personaje(String nombre, int vida)`	Vida menor que 0 se guarda como 0; mayor que 100, como 100	`10`	`"Brakus | vida 80"`
`Caballero` extends `Personaje`	`Caballero(String nombre, int vida, int fuerza)`		Lo del padre más la fuerza. **Usa `super.atacar()`**	Lo del padre más `" | fuerza 15"`
`Paladin` extends `Caballero`	`Paladin(String nombre, int vida, int fuerza, int fe)`		Lo del padre más la mitad de la fe (división entera). **Usa `super.atacar()`**	Lo del padre más `" | fe 40"`

`Mago` extends `Personaje`	`Mago(String nombre, int vida, int mana)`	Mana menor que 0 se guarda como 0	La mitad del mana. **No** usa el del padre	Lo del padre más `" | mana 50"`

Con las cuatro listas, agrega `MainPersonajes.java` y córrelo:

```
public class MainPersonajes {
    public static void main(String[] args) {
        Personaje aldeano = new Personaje("Aldeano", 150);
        Caballero g = new Caballero("Brakus", 80, 15);
        Paladin p = new Paladin("Lunara", -5, 12, 40);
        Mago m = new Mago("Zafiro", 60, 50);

        System.out.println(aldeano + " ataca con " +
aldeano.atacar());
        System.out.println(g + " ataca con " + g.atacar());
        System.out.println(p + " ataca con " + p.atacar());
        System.out.println(m + " ataca con " + m.atacar());
        System.out.println(p.getNombre() + " tiene fuerza " +
p.getFuerza() + " y fe " + p.getFe());
    }
}
```

DEBE SALIR

```
Aldeano | vida 100 ataca con 10
Brakus | vida 80 | fuerza 15 ataca con 25
Lunara | vida 0 | fuerza 12 | fe 40 ataca con 42
Zafiro | vida 60 | mana 50 ataca con 25
Lunara tiene fuerza 12 y fe 40
```

El 42 de Lunara se arma en cadena: `Personaje` da 10, `Caballero` le suma su fuerza (10 + 12 = 22) y `Paladin` le suma la mitad de su fe (22 + 20 = 42). Cada clase solo sabe sumar lo suyo.

Parte 3 · El orden de construcción, para la bitácora

1. Agrega **al final** del constructor de `Personaje`, de `Caballero` y de `Paladin` un `System.out.println` que diga `"Personaje listo"`, `"Caballero listo"` y `"Paladin listo"`.
2. Corre `MainPersonajes` y toma la captura.
3. Contesta en la bitácora, con tus palabras: **¿en qué orden salen las líneas al crear a Lunara, y por qué?** ¿Por qué al crear a Brakus salen dos líneas y al crear a Lunara, tres?
4. Quita los tres `println` antes de entregar: la salida que se califica es la de la parte 2.

Lo que más truena

No compila

- **`constructor Personaje in class Personaje cannot be applied to given types`**: a tu `Caballero` le falta `super(nombre, vida);`. Si no lo escribes, Java pone `super()` sin argumentos, y `Personaje` no tiene ese constructor.
- **`call to super must be first statement in constructor`**: el `super(...)` tiene que ser la primera línea del constructor, antes de guardar lo tuyo.
- **`nombre has private access in Personaje`**: quisiste usar `nombre` desde `Caballero`. Es privado del padre: usa `super.toString()` o `getNombre()`. Lo mismo con `fuerza` desde `Paladin`: usa `super.atacar()`.
- **`method does not override or implement a method from a supertype`**: escribiste mal el nombre (`tostring`, `Atacar`) y el `@Override` te avisó. Para eso está.

Compila, pero la salida no es la de la guía

- **A Lunara le falta `| fe 40`**: tu `toString` de `Paladin` está mal escrito y no tiene `@Override`, así que se usa el del padre sin avisar. Ponle `@Override` y verás el error.
- **Lunara ataca con 20 en vez de 42**: tu `atacar()` de `Paladin` no llama a `super.atacar()`.
- **El Aldeano sale con vida 150**: el constructor de `Personaje` no valida.

La entrega, paso a paso

```
git checkout entregas_apellido_nombre
git pull origin main
mkdir -p entregas/apellido_nombre/p07
# copia aquí los diez .java, todos los .class y la bitácora en PDF
git add entregas/apellido_nombre/p07
git status
git commit -m "P7: herencia, primera parte"
git push
```

En IntelliJ, los `.class` quedan en `out/production/`. Súbelos todos. Si no aparecen en `git status`, usa `git add -f`.

Modelos de Programación Orientada a Objetos 2027-1 · Grupo 6 · Facultad de Ingeniería, UNAM

Prof. Oscar Manuel Ruiz Hurtado · Práctica 7 · Herencia, primera parte (MADO-21, guía 7) · octubre de 2026